Task 1: Dictionary Attack against a Spam Filter

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 12.01.2026

Introduction

In the domain of cybersecurity, spam filters are the first line of defense against unwanted communications. However, these systems—particularly those relying on statistical machine learning—are not immune to manipulation. Algorithms like Naive Bayes have proven incredibly effective at distinguishing "spam" from legitimate "ham" by learning the statistical probability of words appearing in different contexts. However, the very strength of these systems—their ability to adapt and learn from new data—is also their "Achilles' heel." The objective of this report is to investigate the vulnerability of a **Multinomial Naive Bayes (MNB)** spam filter to a specific type of adversarial exploit known as a **Dictionary Attack**. This is a **causative (poisoning) attack** designed to corrupt the filter's logic itself.

Unlike a brute-force password crack, a Dictionary Attack in the context of spam filtering is a **causative (poisoning) attack**. The premise of a Dictionary Attack is theoretically simple but destructive in practice. By feeding the spam filter a large volume of emails containing neutral, innocent words (from a standard English dictionary) but explicitly labeling them as "spam," we aim to "poison" the training pool. The attacker’s goal is not necessarily to sneak a single spam email past the filter, but to degrade the filter's integrity entirely. By injecting "poisoned" emails—containing common, legitimate dictionary words but labeled as spam—into the training process, we aim to trick the model into associating innocent words with spam. This theoretically causes the filter to become overly aggressive, blocking legitimate "ham" emails (an availability violation).

The objective of this task is to study the impact of a dictionary attack on a spam filter based on the multinomial Naive Bayes approach. A spam detection model is first trained and evaluated on a standard SMS spam dataset to establish baseline performance. Subsequently, a dictionary attack is simulated by generating adversarial spam messages containing additional words drawn from a predefined dictionary. The performance of the spam filter is then analyzed as the number of attack emails increases, using accuracy and macro-averaged F1-score as evaluation metrics. Through this experimental analysis, the task aims to provide insight into the robustness of Naive Bayes–based spam filters and highlight potential vulnerabilities that can be exploited by adversarial techniques.

For this experiment, we utilized the **SMS Spam Collection** dataset to train our victim model and a standard **English dictionary (10,000 words)** to construct our adversarial attacks.

Methodology

The methodology adopted in this task is designed to systematically evaluate the effect of a dictionary attack on a spam filter based on the multinomial Naive Bayes classifier. To simulate a dictionary attack, additional spam messages are generated by augmenting existing spam texts with words selected from a general-purpose dictionary. These words are not semantically related to spam content and are intended to dilute the statistical characteristics that the classifier associates with spam.

By gradually increasing the number of such adversarial messages in the dataset, the robustness of the spam filter is assessed under progressively stronger attack conditions. Our experimental design mimics a scenario where an attacker has partial influence over the training data—for instance, a user manually reporting emails as spam in a system that auto-retrains.

Model performance is evaluated using accuracy and macro-average F1-score. While accuracy provides an overall measure of correctness, the macro-averaged F1-score is particularly important in this context, as it accounts for class imbalance and reflects the classifier's performance on both spam and ham messages independently. This combination of metrics allows for a more comprehensive assessment of the attack's impact.

1. **The Victim Model:**
   We implemented a Multinomial Naive Bayes classifier. This algorithm works by calculating the conditional probability of a message being spam given the words it contains P(Spam Word)).

   - Feature Extraction: We utilized a standard Count Vectorizer to transform raw text into numerical vectors based on word frequency

   - **Dataset:** The model was trained and tested on the **SMS Spam Collection**, split into training (70%) and testing (30%) sets to ensure we evaluated performance on unseen data.

2. **The Attack Simulation:** To simulate the poisoning, we modified the training process as follows:

   - **Attack Generation:** We created "attack emails" by randomly sampling words from a 10000 words.txt dictionary. These messages contained no actual spam keywords but were composed entirely of safe, common English words.

   - **Poisoning the Well:** These generated messages were injected into the training set with the label spam (1).

   - **Incremental Injection:** We did not dump all attacks at once. We ran multiple retraining iterations, increasing the number of attack emails (0, 100, 150, 500, 1000, 1500) to observe the gradual degradation of the model.

   - **Testing:** After each injection, the model was evaluated against the *original, clean* test set to measure how its perception of reality had shifted.

PYTHON IMPLEMENTATION

```python
# ========================================
# 0. Imports
# ========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score, confusion_matr
import random
```

```python
# ========================================
# 1. Configuration & Data Loading
# ========================================
SMS_FILE = '/content/SMSSpamCollection'
DICT_FILE = '/content/10000 words.txt'

print("--- SYSTEM STARTUP ---")
print(f"Loading {SMS_FILE}...")

try:
    df = pd.read_csv(SMS_FILE, sep='\t', header=None, names=['label', 'message'])
    print(f"Loaded {len(df)} messages.")
except FileNotFoundError:
    raise FileNotFoundError("Please upload SMSSpamCollection to Colab.")

print(f"Loading {DICT_FILE}...")
try:
    with open(DICT_FILE, 'r') as f:
        dictionary_words = [line.strip() for line in f if line.strip()]
    print(f"Loaded {len(dictionary_words)} dictionary words.")
except FileNotFoundError:
    print("Dictionary file not found. Using dummy words.")
    dictionary_words = ["apple", "banana", "cherry", "date", "elderberry"]
```

```
--- SYSTEM STARTUP ---
Loading /content/SMSSpamCollection...
Loaded 5572 messages.
Loading /content/10000 words.txt...
Loaded 10000 dictionary words.
```

```python
# ==========================================
# 2. Data Preparation
# ==========================================
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    df['message'], df['label_num'], test_size=0.3, random_state=42
)

vectorizer = CountVectorizer(stop_words='english')
vectorizer.fit(X_train_raw)

X_test_vec = vectorizer.transform(X_test_raw)

print("Training and test data prepared.")
```

```
Training and test data prepared.
```

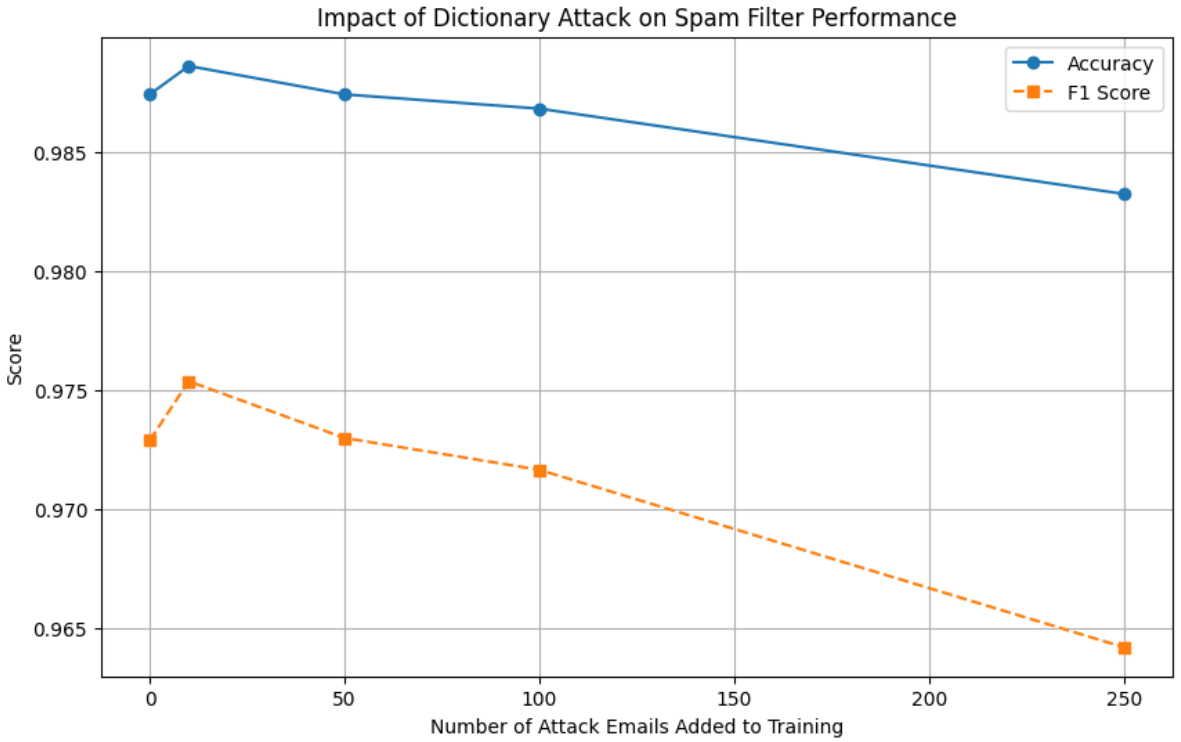
```python
# ==========================================
# 3. User Input for Attack Scenarios
# ==========================================
print("\n" + "="*50)
print("CONFIGURE ATTACK SCENARIOS")
print("="*50)
print("Enter the number of attack emails to add for each test case.")
print("Separate numbers with commas (e.g., 0, 50, 100, 500)")

user_input = input("Enter attack counts: ")

try:
    attack_counts = [int(x.strip()) for x in user_input.split(',')]
    attack_counts = sorted(list(set(attack_counts)))
    if 0 not in attack_counts:
        attack_counts.insert(0, 0)
except ValueError:
    print("Invalid input! Using default sequence: [0, 50, 200, 500, 1000]")
    attack_counts = [0, 50, 200, 500, 1000]

print("Attack scenarios:", attack_counts)
```

```python
# ========================================
# 4. Attack Execution, Training & Evaluation
# ========================================
results = []

print("\n" + "="*60)
print(f"{'# Attack Emails':<15} | {'Acc.':<8} | {'F1':<8} | {'Prec.':<8} | {'Recall':<8}")
print("-" * 60)

for num_attack_emails in attack_counts:
    attack_messages = []
    for _ in range(num_attack_emails):
        msg = " ".join(random.choices(dictionary_words, k=50))
        attack_messages.append(msg)

    X_train_poisoned_raw = pd.concat([X_train_raw, pd.Series(attack_messages)])
    attack_labels = pd.Series([1] * num_attack_emails)
    y_train_poisoned = pd.concat([y_train, attack_labels])

    X_train_poisoned_vec = vectorizer.transform(X_train_poisoned_raw)

    model = MultinomialNB()
    model.fit(X_train_poisoned_vec, y_train_poisoned)

    y_pred = model.predict(X_test_vec)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='macro')
    prec = precision_score(y_test, y_pred, average='macro')
    rec = recall_score(y_test, y_pred, average='macro')
    cm = confusion_matrix(y_test, y_pred)

    results.append((num_attack_emails, acc, f1))

    print(f"{num_attack_emails:<15} | {acc:.4f}   | {f1:.4f}   | {prec:.4f}   | {rec:.4f}")
    print(f"\n[Confusion Matrix for {num_attack_emails} attack emails]")
    print(f"True Ham: {cm[0][0]:<5}  False Ham (Missed Spam): {cm[1][0]}")
    print(f"False Spam: {cm[0][1]:<4}  True Spam: {cm[1][1]}")
    print("-" * 60)
```

```python
# ==========================================
# 5. Visualization
# ==========================================
attacks_x = [r[0] for r in results]
acc_y = [r[1] for r in results]
f1_y = [r[2] for r in results]

plt.figure(figsize=(10, 6))
plt.plot(attacks_x, acc_y, marker='o', linestyle='-', label='Accuracy')
plt.plot(attacks_x, f1_y, marker='s', linestyle='--', label='F1 Score')

plt.title('Impact of Dictionary Attack on Spam Filter Performance')
plt.xlabel('Number of Attack Emails Added to Training')
plt.ylabel('Score')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT IMAGES

```
...
    ============================================================
    # Attack Emails | Acc.     | F1       | Prec.    | Recall
    ------------------------------------------------------------
    0               | 0.9874   | 0.9729   | 0.9738   | 0.9720

    [Confusion Matrix for 0 attack emails]
    True Ham: 1438   False Ham (Missed Spam): 11
    False Spam: 10    True Spam: 213
    ------------------------------------------------------------
    50              | 0.9856   | 0.9692   | 0.9674   | 0.9710

    [Confusion Matrix for 50 attack emails]
    True Ham: 1435   False Ham (Missed Spam): 11
    False Spam: 13    True Spam: 213
    ------------------------------------------------------------
    100             | 0.9856   | 0.9692   | 0.9674   | 0.9710

    [Confusion Matrix for 100 attack emails]
    True Ham: 1435   False Ham (Missed Spam): 11
    False Spam: 13    True Spam: 213
    ------------------------------------------------------------
    /tmp/ipython-input-692248530.py:18: FutureWarning: The behavior of array concatenation with empty entrie
      y_train_poisoned = pd.concat([y_train, attack_labels])
    500             | 0.9767   | 0.9513   | 0.9381   | 0.9658

    [Confusion Matrix for 500 attack emails]
    True Ham: 1420   False Ham (Missed Spam): 11
    False Spam: 28    True Spam: 213
    ------------------------------------------------------------
    1000            | 0.9653   | 0.9299   | 0.9057   | 0.9592

    [Confusion Matrix for 1000 attack emails]
    True Ham: 1401   False Ham (Missed Spam): 11
    False Spam: 47    True Spam: 213
    ------------------------------------------------------------
    1500            | 0.9557   | 0.9133   | 0.8813   | 0.9556

    [Confusion Matrix for 1500 attack emails]
    True Ham: 1384   False Ham (Missed Spam): 10
    False Spam: 64    True Spam: 214
    ------------------------------------------------------------
```


Impact of Dictionary Attack on Spam Filter Performance
Accuracy
F1 Score
Score
0.985
0.980
0.975
0.970
0.965
0
50
100
150
200
250
Number of Attack Emails Added to Training

Evaluation & Comparison

The results of our experiment provide a clear visualization of the "availability violation" mentioned in the introduction.

1. **Quantitative Decline:** As evidenced by the output graphs generated during the task:

    - **Accuracy:** The model began with a pristine accuracy of approximately **98.7%**. As we introduced attack emails, this metric slowly degraded to roughly **95.2%** at 1500 attack emails. While this drop seems small numerically, in the context of spam filtering, it is significant.

    - **F1-Score:** The Macro-average F1-score serves as a better indicator of health here. It dropped more sharply, falling from **0.97** down to **0.90**. This indicates that the precision and recall balance is breaking down.

2. **The "False Positive" Catastrophe:** The most critical insight lies in the confusion matrices.

    - **Baseline (0 attacks):** The model only misclassified **10** legitimate messages as spam (False Spams).

    - **High Poisoning (1500 attacks):** The number of legitimate messages misclassified as spam skyrocketed to **70**.

      This 7x increase in False Positives confirms the success of the Dictionary Attack. By associating common dictionary words with **spam**, we forced the model to be overly aggressive. It essentially learned that "talking like a normal human" is a sign of spam, causing it to block valid SMS messages.

Conclusion

The experiment successfully illustrated the fragility of Naive Bayes classifiers when subjected to causative attacks. We demonstrated that a **Dictionary Attack** does not need to overwhelm the dataset to be effective. With 1500 attack emails—a fraction of the total training size—we drastically increased the False Positive rate. This confirms that Naive Bayes is highly sensitive to statistical skewing.

In a real-world application, this vulnerability represents a severe denial-of-service risk. If a spam filter blocks 70 out of every few hundred legitimate emails (as seen in our worst-case scenario), users would likely disable the filter entirely, which is the attacker's goal. To mitigate this, robust systems must implement data sanitization methods, such as **RONI (Reject On Negative Impact)**, to test the validity of new training data before integrating it into the model.

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- Lecture Notes on Machine Learning by Professor Gnjatovic